

Sistema de Gestión de Tienda de Ropa

Descripción del Proyecto

Este proyecto es una aplicación de escritorio desarrollada en Java utilizando el framework Swing para gestionar una tienda de ropa. Permite administrar clientes, productos y ventas de manera sencilla e intuitiva a través de una interfaz gráfica de usuario.

Arquitectura

El proyecto sigue el patrón de arquitectura Modelo-Vista-Controlador (MVC) con las siguientes capas:

- **Modelo (Model):** Representa las entidades del negocio (Cliente, Producto, Venta, Detalle de Venta).
- **Vista (View):** Componentes de la interfaz gráfica (paneles Swing).
- **Controlador (Controller):** Servicios que manejan la lógica de negocio.
- **Repositorio (Repository):** Capa de acceso a datos (actualmente en memoria).

Tecnologías Utilizadas

- **Lenguaje:** Java 17
- **Framework de UI:** Swing
- **Gestión de Dependencias:** Maven
- **IDE Recomendado:** NetBeans (por los archivos .form)

Estructura del Proyecto

```
TIENDA_DE_ROPA/  
├── pom.xml  
├── src/  
│   ├── main/  
│   │   ├── java/  
│   │   │   ├── com/  
│   │   │   │   ├── grupo4/  
│   │   │   │   │   ├── manageclient/  
│   │   │   │   │   │   ├── MainFrame.java  
│   │   │   │   │   │   ├── ManageClient.java  
│   │   │   │   │   │   ├── model/  
│   │   │   │   │   │   │   ├── Client.java  
│   │   │   │   │   │   │   ├── Product.java  
│   │   │   │   │   │   │   ├── Sale.java  
│   │   │   │   │   │   │   └── SaleDetail.java  
│   │   │   │   │   │   ├── repository/  
│   │   │   │   │   │   │   ├── ClientRepository.java  
│   │   │   │   │   │   │   ├── IClientRepository.java  
│   │   │   │   │   │   │   ├── IProductRepository.java  
│   │   │   │   │   │   │   ├── ISaleRepository.java  
│   │   │   │   │   │   │   ├── ProductRepository.java  
│   │   │   │   │   │   │   └── SaleRepository.java  
│   │   │   │   │   │   └── service/  
│   │   │   │   │   └── service/
```

```
├── ClientService.java
├── IClientService.java
├── IProductService.java
├── ISaleService.java
├── ProductService.java
├── SaleService.java
├── view/
│   ├── ClientPanel.form
│   ├── ClientPanel.java
│   ├── ProductPanel.form
│   ├── ProductPanel.java
│   ├── SalePanel.form
│   └── SalePanel.java
└── target/
    └── ...
```

Modelos de Datos

Client

Representa un cliente de la tienda.

Atributos:

- **idClient** (int): Identificador único del cliente
- **nameClient** (String): Nombre del cliente
- **email** (String): Correo electrónico
- **phoneNumber** (String): Número de teléfono

Product

Representa un producto de la tienda.

Atributos:

- **idProduct** (int): Identificador único del producto
- **nameProduct** (String): Nombre del producto
- **size** (String): Talla del producto
- **color** (String): Color del producto
- **unitPrice** (double): Precio unitario
- **stock** (int): Cantidad en inventario

Sale

Representa una venta realizada.

Atributos:

- **idSale** (int): Identificador único de la venta
- **client** (Client): Cliente que realizó la compra
- **details** (List): Lista de detalles de la venta

- `total` (double): Total calculado de la venta

SaleDetail

Representa el detalle de una venta (producto específico vendido).

Atributos:

- `product` (Product): Producto vendido
- `quantity` (int): Cantidad vendida
- `unitPrice` (double): Precio unitario al momento de la venta
- `subtotal` (double): Subtotal calculado (cantidad * precio unitario)

Repositorios

Los repositorios manejan el acceso a datos. Actualmente implementan almacenamiento en memoria usando `ArrayList`.

ClientRepository

- `save(Client client)`: Guarda un nuevo cliente
- `getAllClients()`: Obtiene todos los clientes
- `findById(int idClient)`: Busca cliente por ID
- `update(Client updateClient)`: Actualiza un cliente existente
- `delete(int idClient)`: Elimina un cliente por ID

ProductRepository

- `save(Product product)`: Guarda un nuevo producto
- `getAllProducts()`: Obtiene todos los productos
- `findById(int idProduct)`: Busca producto por ID
- `update(Product updateProduct)`: Actualiza un producto existente
- `delete(int idProduct)`: Elimina un producto por ID

SaleRepository

- `save(Sale sale)`: Guarda una nueva venta
- `getAllSales()`: Obtiene todas las ventas
- `findById(int idSale)`: Busca venta por ID
- `update(Sale updateSale)`: Actualiza una venta existente
- `delete(int idSale)`: Elimina una venta por ID

Servicios

Los servicios contienen la lógica de negocio y validaciones.

ClientService

Validaciones:

- ID debe ser positivo

- Nombre obligatorio
- Email obligatorio y con formato válido
- Teléfono obligatorio, solo números, 7-10 dígitos

Métodos:

- `registerClient(int id, String name, String email, String phone)`: Registra nuevo cliente
- `getAllClients()`: Obtiene todos los clientes
- `findClientById(int id)`: Busca cliente por ID
- `updateClient(int id, String name, String email, String phone)`: Actualiza cliente
- `deleteClient(int id)`: Elimina cliente

ProductService

Validaciones:

- ID debe ser positivo
- Nombre obligatorio
- Precio debe ser positivo
- Stock no puede ser negativo

Métodos:

- `registerProduct(int id, String name, String size, String color, double price, int stock)`: Registra nuevo producto
- `getAllProducts()`: Obtiene todos los productos
- `findProductById(int id)`: Busca producto por ID
- `updateProduct(int id, String name, String size, String color, double price, int stock)`: Actualiza producto
- `deleteProduct(int id)`: Elimina producto

SaleService

Validaciones:

- Cliente debe existir
- Productos deben existir y tener stock suficiente
- Cantidad debe ser positiva

Métodos:

- `registerSale(int clientId, List<SaleDetail> details)`: Registra nueva venta
- `getAllSales()`: Obtiene todas las ventas
- `findSaleById(int id)`: Busca venta por ID
- `updateSale(int id, int clientId, List<SaleDetail> details)`: Actualiza venta
- `deleteSale(int id)`: Elimina venta

Vistas

MainFrame

Ventana principal de la aplicación con pestañas para:

- Clientes
- Productos
- Ventas

ClientPanel

Panel para gestión de clientes:

- Formulario para agregar/editar clientes
- Tabla para mostrar lista de clientes
- Botones para operaciones CRUD

ProductPanel

Panel para gestión de productos:

- Formulario para agregar/editar productos
- Tabla para mostrar lista de productos
- Botones para operaciones CRUD

SalePanel

Panel para gestión de ventas:

- Selección de cliente
- Selección de productos y cantidades
- Cálculo automático de totales
- Tabla para mostrar lista de ventas

Instalación y Ejecución

Prerrequisitos

- Java 17 o superior
- Maven 3.6+

Compilación

```
mvn clean compile
```

Ejecución

```
mvn exec:java
```

O directamente desde el IDE:

```
java -cp target/classes com.grupo4.manageclient.ManageClient
```

Limitaciones y Mejoras Futuras

Limitaciones Actuales

- **Persistencia:** Los datos se almacenan solo en memoria, se pierden al cerrar la aplicación
- **Concurrencia:** No maneja múltiples usuarios simultáneamente
- **Validaciones:** Limitadas, podrían expandirse
- **UI:** Interfaz básica, podría mejorarse con más funcionalidades

Mejoras Sugeridas

- Integrar base de datos (MySQL, PostgreSQL)
- Agregar autenticación de usuarios
- Implementar reportes y estadísticas
- Mejorar la interfaz de usuario
- Agregar manejo de inventario avanzado
- Implementar backup y restauración de datos

Contribuidores

- Angel (modelos y servicios de cliente)
- Samuel (modelos y servicios de venta)

Licencia

Este proyecto está bajo la Licencia MIT. Ver archivo LICENSE para más detalles.